

GusAI RAG Capstone: Technical Documentation

October 2026

Abstract

Gus is a full-stack Retrieval-Augmented Generation (RAG) system designed to let users query their own documents through natural language. The system accepts PDF, Word, Notion, and web-page inputs, chunks and embeds them with Voyage AI, and stores them in a PostgreSQL/pgvector database. Queries are answered by retrieving the most relevant chunks and feeding them, along with conversation history, to a Groq-hosted LLM.

The project ships two retrieval strategies. v1 is a top-k cosine similarity baseline. v2 is a five-stage pipeline: LLM-driven query decomposition for multi-hop questions, hybrid search (dense MMR + BM25 keyword), cross-session deduplication, Cohere neural reranking, and contextual compression. On top of v2 retrieval, an agentic layer built with LangChain's `create_agent` wraps retrieval as a callable tool, allowing the model to retrieve iteratively and cite sources explicitly.

Retrieval quality was benchmarked using RAGAS on a 15-question dataset drawn from AI course lecture slides. v2 outperforms v1 across all tested k values. At k=7, Faithfulness improves from 0.619 to 0.818 and Context Precision from 0.703 to 0.851. The agentic layer is projected to improve Faithfulness further to approximately 0.910 at k=7, pending a post-integration re-run of the evaluation harness.

The backend is a FastAPI service deployed on Render with a PostgreSQL database on Neon. CI/CD is handled by GitHub Actions: a testing workflow runs on pull requests into development branches, and a deploy workflow triggers a Render webhook on merge to main. The system is intentionally scoped as a single-session, no-authentication application appropriate for a portfolio demonstration; the primary known limitations absent auth, BM25 full-table-scan scaling, and the gap between actual and estimated agentic eval scores are documented explicitly.

Table of Contents

1. Architecture Overview	3
1.1 High-Level Stack	3
1.2 Ingestion Path (end to end)	4
1.3 Query Path — v1 (Baseline)	4
1.4 Query Path — v2 (Full Pipeline)	5
2. Module Reference	6
2.1 config.py — Settings	6
2.2 vectorstore.py — Storage Layer	7
2.3 ingestion.py — Document Loading and Chunking	7
2.4 retrieval.py — Retrieval Pipeline	8
v1 — Baseline	8
v2 — Full Pipeline (five stages)	8
BM25 Cache	9
2.5 generation.py — Generation Layer	9
v1 — generate()	9
v2 — run_agent()	9
Prompt Templates	9
2.6 chat_history.py — Session History	10
2.7 documents.py (Router) — Document Endpoints	10
2.8 query.py (Router) — Query Endpoint	10
2.9 schemas.py — Pydantic Models	11
2.10 main.py — Application Entry Point	11
3. Evaluation Harness and Results	12
3.1 Harness — eval/run_eval.py and eval/dataset.py	12
3.2 Results	12
v1 Baseline (actual — run 25 September 2026)	12
v2 Pipeline (actual — before agentic layer was added)	12
v2 + Agentic Layer (estimated — not yet measured)	13
Key Observations	13
4. Design Decisions	14
5. Known Limitations	16
6. Setup and Configuration	17
6.1 Prerequisites	17
6.2 Environment Variables (.env in backend/)	17
6.3 Dependencies	17
6.4 Running Locally	18
6.5 CI/CD Workflows	19
Appendix A — Evaluation Dataset	20
Lecture 1 — Introduction to AI	20
Lecture 2 — Search Algorithms	20
Lecture 3 — Adversarial Search	21
Lecture 4 — Knowledge Representation and Reasoning	21
Lecture 5 — Matrix Games and Logic Applications	22
Appendix B — API Quick Reference	23
Document Endpoints	23
Query Endpoint	23
Health / Root	23
Error Codes	24

1. Architecture Overview

Gus is a Retrieval-Augmented Generation (RAG) system built with FastAPI and LangChain. Users upload documents or supply web URLs; the system chunks them, embeds them with Voyage AI, and stores them in a PostgreSQL/pgvector database. Queries are answered by retrieving the most relevant chunks and feeding them to a Groq-hosted LLM.

Two FastAPI routers are provided: /documents for ingestion and management, and /query for question answering.

1.1 High-Level Stack

Layer	Technology
Web framework	FastAPI + Uvicorn
LLM provider	Groq (ChatGroq; default model: openai/gpt-oss-120b)
Embeddings	Voyage AI — VoyageAIEmbeddings, voyage-4-lite
Vector store	PostgreSQL + pgvector via LangChain-Postgres (PGVector)
Chat history	PostgreSQL via langchain_postgres.PostgresChatMessageHistory
BM25 sparse search	rank_bm25 via LangChain-Community BM25Retriever (in-memory, cached)
Reranker	Cohere Rerank — rerank-v3.5
File loaders	PyPDFLoader, Docx2txtLoader, NotionDirectoryLoader, trafilatura (web)
Deploy	Render — auto-deploy via webhook on merge to main

1.2 Ingestion Path (end to end)

What happens when a user uploads a file or submits a URL:

- 1. Client sends a multipart POST to /documents/upload (file) or JSON to /documents/web (URL).
- 2. documents.py dispatches to the correct loader in ingestion.py based on file extension: .pdf → PyPDFLoader, .docx → Docx2txtLoader, .zip → NotionDirectoryLoader, URL → trafilatura.
- 3. ingestion.py loads the raw content into LangChain Document objects and runs RecursiveCharacterTextSplitter (chunk_size=1500, chunk_overlap=200).
- 4. The router stamps each chunk with source, source_type, and session_id metadata, then calls vectorstore.add_documents().
- 5. vectorstore.py sends chunks to VoyageAI for embedding and writes them to langchain_pg_embedding in batches (INGESTION_BATCH_SIZE, default 100).
- 6. The BM25 in-memory cache for the session is invalidated so the next query rebuilds the keyword index.
- 7. An IngestionResponse is returned: source name, type, chunk count, sample chunk (first 200 chars), and session_id.

1.3 Query Path — v1 (Baseline)

v1 is the simple single-pass pipeline and is the default (RETRIEVAL_STRATEGY = "v1"):

- 1. Client POSTs { question, version: "v1", session_id } to /query/.
- 2. query.py loads the last 30 messages from PostgresChatMessageHistory for the session.
- 3. retrieval.retrieve() calls retrieve_v1(): plain top-k similarity search via PGVector.as_retriever(), filtered by session_id.
- 4. generation.generate() stuffs the retrieved chunk texts into PROMPT_TEMPLATE and calls ChatGroq in one pass.
- 5. The answer and deduplicated sources are returned; both the user message and AI answer are appended to chat history.

1.4 Query Path — v2 (Full Pipeline)

v2 replaces the single similarity search with a multi-stage retrieval pipeline wrapped in an agentic loop:

- 1. Client POSTs { question, version: "v2", session_id } to /query/.
- 2. query.py loads history and calls generation.run_agent() instead of generate().
- 3. run_agent() invokes a LangChain create_agent agent. The agent reads AGENT_PROMPT and decides whether to call the retrieve_documents tool.
- 4. retrieve_documents (@tool) calls retrieval.retrieve(question, version="v2", session_id=...). The session_id travels through AgentContext, not the model's message context, preventing prompt injection.
- 5. Inside retrieve_v2, five stages run in sequence:
 - a. Decomposition — decompose_question() asks the LLM to split multi-hop questions into up to 3 sub-questions (or return the original if single-hop).
 - b. Hybrid retrieval — SelfQueryRetriever (dense, MMR, natural-language metadata filtering) + BM25Retriever (sparse) via EnsembleRetriever. Each sub-question is run through the ensemble; results are pooled.
 - c. Session guard — every candidate's session_id metadata is re-checked against the request session_id. Chunks from other sessions are dropped.
 - d. Deduplication + reranking — dedupe() removes (source, content) duplicates; rerank() scores candidates with Cohere and keeps the top k.
 - e. Contextual compression — compress() uses LLMChainExtractor to trim each chunk to its most relevant sentences.
- 6. The tool returns (text_summary, [Document]). Text goes to the model; Documents ride on ToolMessage.artifact for source attribution.
- 7. The agent may call retrieve_documents up to MAX_RETRIEVALS (3) times (enforced by ToolCallLimitMiddleware; exit_behavior="continue").
- 8. run_agent() collects all ToolMessage artifacts from the turn, deduplicates sources, and returns { answer, sources }.
- 9. query.py saves history and returns QueryResponse.

2. Module Reference

2.1 config.py — Settings

Loads all configuration from environment variables (and a .env file) via pydantic-settings BaseSettings. A module-level singleton settings is imported everywhere else.

Setting	Default	Purpose
GROQ_API_KEY	""	Groq API key for LLM and retrieval LLM calls
LLM_MODEL	"openai/gpt-oss-120b"	Groq model string passed to ChatGroq
DATABASE_URL	""	PostgreSQL connection string (psycopg3 dialect)
ALLOWED_ORIGINS	["http://localhost:3000"]	CORS allowed origins list
COHERE_API_KEY	""	Cohere API key for rerank-v3.5
VOYAGE_API_KEY	""	Voyage AI API key for embeddings
VOYAGE_MODEL	"voyage-4-lite"	Voyage AI embedding model
INGESTION_BATCH_SIZE	100	Chunk batch size for vectorstore writes
DEFAULT_K	7	Default number of chunks returned from retrieval
RETRIEVAL_STRATEGY	"v1"	Active pipeline: "v1" or "v2". Flip to "v2" to enable the full pipeline.
RETRIEVAL_FETCH_K_MULTIPLIER	4	v2 over-fetches k x this many candidates before reranking down to k
BM25_WEIGHT	0.4	BM25 share of hybrid ensemble (vector gets 1 - BM25_WEIGHT = 0.6)
RERANK_MODEL	"cross-encoder/ms-marco..."	Defined but not used at runtime — Cohere is hardcoded in retrieval.py. See Known Limitations.
MULTI_HOP_MAX_SUBQUESTIONS	3	Maximum sub-questions the decomposer may generate

2.2 vectorstore.py — Storage Layer

Owns the PGVector connection, VoyageAI embedding model, and all raw database operations. Nothing outside this module writes to or reads from the vector table directly.

Key module-level objects:

- **embeddings** — VoyageAIEmbeddings instance. All embedding calls go through this; shared with the PGVector store.
- **vector_store** — PGVector instance. Collection name: "documents". create_extension=False (pgvector assumed pre-installed).
- **engine** — SQLAlchemy engine with pool_pre_ping=True, pool_recycle=300s. DATABASE_URL is normalised to postgresql+psycopg:// dialect required by psycopg3.

Function	Description
add_documents(chunks)	Writes chunks in INGESTION_BATCH_SIZE batches via PGVector.add_documents(). Returns list of generated row IDs.
ensure_indexes()	Runs at module load. Creates two partial indexes on langchain_pg_embedding: one on (collection_id, source), one on (collection_id, session_id). Uses CREATE INDEX IF NOT EXISTS — safe to run repeatedly.
delete_by_source(source_name, session_id)	Raw SQL DELETE filtered by collection name, source metadata field, and optionally session_id. Returns deleted_count and collection_id.
list_documents(session_id)	GROUP BY query returning source, source_type, session_id, and chunk_count per distinct source.
get_all_documents(session_id)	Full table scan returning all Document objects for BM25 index construction. A code comment flags that this should become paginated/cached if the corpus grows large.

2.3 ingestion.py — Document Loading and Chunking

Handles loading raw content from all supported source types and splitting it into fixed-size chunks. No knowledge of the vectorstore or session state — produces only lists of LangChain Documents.

Function	Input	Loader	Notes
ingest_pdf(file_bytes, filename)	Raw bytes	PyPDFLoader	Writes to a temp file (deleted after load). Restores original filename in metadata because PyPDFLoader overwrites it with the temp path.
ingest_docx(file_bytes, filename)	Raw bytes	Docx2txtLoader	Same temp-file pattern as PDF. One Document per file before chunking.

Function	Input	Loader	Notes
ingest_web(url)	URL string	trafilatura.extract()	Streams response in 8 KB chunks, capped at 10 MB via fetch_bounded(). Raises ValueError if trafilatura returns no text.
ingest_notion(zip_bytes, filename)	Raw bytes	NotionDirectoryLoader	Extracts ZIP to a temp directory, loads all Markdown files. Raises ValueError if no .md files found.
fetch_bounded(url)	URL string	requests (streaming)	Internal. Enforces byte cap via Content-Length header check and streaming accumulation. Raises ValueError on network errors or size limit exceeded.
chunk_documents(docs)	Doc list	RecursiveCharacterTextSplitter	chunk_size=1500, chunk_overlap=200. Called by every ingest_* function as the final step.

2.4 retrieval.py — Retrieval Pipeline

Implements both retrieval strategies. The public entry point is `retrieve()`, which dispatches to `retrieve_v1()` or `retrieve_v2()` based on the version argument or the `RETRIEVAL_STRATEGY` config value.

v1 — Baseline

`retrieve_v1()` uses `PGVector.as_retriever()` with plain top-k similarity search filtered by `session_id`. No reranking or hybrid search. Used as the eval baseline and as the default API strategy.

v2 — Full Pipeline (five stages)

- **Decomposition.** `decompose_question()` sends the question to the LLM with a zero-shot prompt. Single-hop: LLM returns the original question unchanged. Multi-hop: LLM returns 2–3 sub-questions, one per line. Parsing strips leading bullets/asterisks/spaces and drops empty lines.
- **Hybrid retrieval.** `SelfQueryRetriever` (dense, MMR, with natural-language metadata filtering) + `BM25Retriever` (sparse) combined via `EnsembleRetriever` (weights: 60% vector, 40% BM25 by default). Each sub-question is invoked separately; results are pooled.
- **Session guard.** Every candidate's `session_id` metadata is re-checked against the request `session_id` after retrieval. Belt-and-suspenders: the `SelfQueryRetriever` already has a filter applied, but this catches any leakage.
- **Deduplication + reranking.** `dedupe()` removes (source, page_content) duplicates. `rerank()` sends the candidates to Cohere rerank-v3.5 and returns the top k. Returns early if docs is empty.
- **Contextual compression.** `compress()` uses `LLMChainExtractor` (same retrieval LLM) to extract only the sentences from each chunk that are relevant to the question. Falls back to uncompressed chunks if output is empty.

BM25 Cache

The BM25 index is built from a full scan of the vector table (`get_all_documents`). The result is cached per `session_id` in an `OrderedDict` with:

- TTL of 600 seconds (`BM25_CACHE_TTL_SECONDS`). After expiry, the next query triggers a rebuild.
- LRU eviction at 20 sessions (`BM25_CACHE_MAX_SESSIONS`).
- Immediate invalidation via `invalidate_bm25_cache(session_id)`, called by the documents router after every ingestion or deletion.

2.5 generation.py — Generation Layer

Provides two generation modes sharing the same ChatGroq LLM instance (`temperature=0`).

v1 — `generate()`

Builds a system message from `PROMPT_TEMPLATE` with retrieved context interpolated, then calls `llm.invoke([system_message, *history, human_message])`. Returns the string content of the response. Stateless — no agent, no tool calls.

v2 — `run_agent()`

Invokes the `create_agent` agent built by `get_agent()` (cached with `@lru_cache` on first use). Key design points:

- **AgentContext** carries `session_id` per-request. Passed as `context=` to `agent.invoke()`, accessible inside `retrieve_documents` via `ToolRuntime[AgentContext]`. The model never sees the `session_id` in its message context.
- **ToolCallLimitMiddleware** caps `retrieve_documents` calls at `MAX_RETRIEVALS=3`. `exit_behavior="continue"` means the agent produces a best-effort answer on hitting the cap rather than erroring.
- **Source attribution**: after the agent returns, `run_agent()` iterates over only the messages produced during the current turn (sliced from the input history length). Document objects are collected from `ToolMessage.artifact` fields and deduplicated via `dedupe_sources()`.

Prompt Templates

Both `PROMPT_TEMPLATE` (v1) and `AGENT_PROMPT` (v2) share the same persona (Gus, built by Gio) and the same two-branch instruction structure:

- Small talk / questions about Gus: answer directly, ignore context entirely, no source sections.
- Document questions: ground answer in retrieved context only; if context is incomplete, add a clearly separate section for outside knowledge; never blend outside knowledge into the grounded part.

The v2 agent prompt also includes explicit instructions for tool-call failure: if retrieval reports failure or the search limit was reached, say so plainly rather than presenting an ungrounded answer.

2.6 chat_history.py — Session History

Thin wrapper around LangChain's `PostgresChatMessageHistory`. At module import time it opens a one-off psycopg connection to run `create_tables()` (creating the `chat_history` table if absent) and immediately closes it.

The public API is the context manager `get_history(session_id)`, which opens a psycopg connection, yields a `PostgresChatMessageHistory` instance, and closes the connection in the finally block. `query.py` calls it twice per request: once to read history before generation, and once to write the new turn after.

Note: the table creation runs synchronously at import time. If `DATABASE_URL` is misconfigured, the application fails on startup rather than on the first query — intentional, to surface configuration errors early.

2.7 documents.py (Router) — Document Endpoints

FastAPI `APIRouter` mounted at `/documents/`. Uses `LOADER_DISPATCH`, a dict mapping file extensions to `(source_type, loader_fn)` tuples, to dispatch uploads. The `.zip` extension is exclusively assumed to be a Notion export.

Endpoint	Method	Description
GET <code>/documents/</code>	GET	Lists all documents for a <code>session_id</code> (optional query param). Returns <code>source</code> , <code>source_type</code> , <code>session_id</code> , <code>chunk_count</code> per document.
POST <code>/documents/upload</code>	POST	Multipart form upload. Accepts <code>.pdf</code> , <code>.docx</code> , <code>.zip</code> . <code>session_id</code> is a <code>Form()</code> field (form-encoded, not JSON, to match the multipart format). Returns <code>IngestionResponse</code> .
POST <code>/documents/web</code>	POST	JSON body with <code>url</code> (<code>HttpUrl</code>) and optional <code>session_id</code> . Returns <code>IngestionResponse</code> .
DELETE <code>/documents/{source_name:path}</code>	DELETE	Deletes all chunks for the named source. The <code>:path</code> converter allows slashes in source names (e.g. URLs). Returns HTTP 404 if no chunks found.

`store_chunks()` is an internal helper that stamps `source_type` and `session_id` on chunks, calls `vectorstore.add_documents()`, and invalidates the BM25 cache. Embedding failures raise HTTP 500; ingestion parse errors (`ValueError` from `ingestion.py`) raise HTTP 422.

2.8 query.py (Router) — Query Endpoint

Single POST `/query/` endpoint. Accepts `QueryRequest` (`question`, `version`, optional `session_id`). Generates a `session_id` if none is supplied. Routes to v1 or v2 generation, saves history, returns `QueryResponse` (`answer`, `sources`, `session_id`).

`MAX_HISTORY_MESSAGES = 30`. History is sliced to the last 30 messages before being passed to generation; all messages are retained in the database.

2.9 schemas.py — Pydantic Models

Model	Fields	Used by
WebIngestRequest	url: HttpUrl, session_id: str None	POST /documents/web request body
DocumentSummary	source, source_type, session_id, chunk_count	GET /documents/ response
IngestionResponse	source, source_type, chunk_count, sample_chunk, session_id	POST /documents/upload and /web response
QueryRequest	question: str, version: Literal["v1", "v2"] = "v1", session_id: str None	POST /query/ request body
SourceInfo	source: str, source_type: str None	Nested in QueryResponse
QueryResponse	answer: str, sources: list[SourceInfo], session_id: str	POST /query/ response

2.10 main.py — Application Entry Point

Creates the FastAPI app, adds CORSMiddleware (origins from settings.ALLOWED_ORIGINS), and includes the documents and query routers. Provides GET / (welcome message) and GET /health. Swagger UI is available at /docs when the server is running.

3. Evaluation Harness and Results

3.1 Harness — `eval/run_eval.py` and `eval/dataset.py`

The evaluation harness compares v1 and v2 on a fixed 15-question dataset drawn from 5 AI lecture PDFs (3 questions per lecture). Topics: AI introduction, search algorithms, adversarial search, knowledge representation, and matrix games.

Two RAGAS metrics are measured:

- **Faithfulness:** whether the generated answer is supported by the retrieved context. Range 0–1.
- **LLMContextPrecisionWithReference:** whether relevant retrieved chunks are ranked before irrelevant ones. Range 0–1.

Limitation: the same ChatGroq model is used both as the system under test and as the RAGAS judge. This introduces known self-evaluation bias. Scores should be treated as directional comparisons between versions, not absolute quality benchmarks.

To run: `python eval/run_eval.py v1 --k 7` (replace v1/7 as needed). Results are saved as JSON to `eval/results/` with a timestamp in the filename.

3.2 Results

v1 Baseline (actual — run 25 September 2026)

k	Faithfulness	Context Precision
4	0.5565	0.6722
7	0.6190	0.7031
10	0.5991	0.6387

v2 Pipeline (actual — before agentic layer was added)

k	Faithfulness	Context Precision
4	0.6928	0.8537
7	0.8184	0.8514
10	0.7418	0.8514

v2 + Agentic Layer (estimated — not yet measured)

The actual v2 scores above were produced before the `create_agent / ToolCallLimitMiddleware` layer was added. The following estimates project the expected uplift:

k	Estimated Faithfulness	Estimated Context Precision
4	~0.8400 (up from 0.6928)	~0.8750 (up from 0.8537)
7	~0.9100 (up from 0.8184)	~0.8800 (up from 0.8514)
10	~0.8600 (up from 0.7418)	~0.8650 (up from 0.8514)

These estimates are inferred, not measured. A re-run of the eval harness against the full v2+agent stack is needed to confirm them.

Key Observations

- v2 consistently outperforms v1 on both metrics across all k values. The largest absolute gain is Faithfulness at k=7: +0.1994 (0.6190 → 0.8184).
- k=7 is the sweet spot for v2 (highest Faithfulness, competitive Context Precision). This is consistent with `DEFAULT_K=7` in `config.py`.
- v1 Context Precision drops sharply at k=10 (0.7031 → 0.6387), suggesting that adding more candidates without reranking introduces noise. v2 Context Precision is stable at k=7 and k=10 (both 0.8514), confirming reranking absorbs the noise.
- Faithfulness falls from k=7 to k=10 in both versions, possibly from context dilution: more chunks means more off-topic content for the LLM to partially follow.

4. Design Decisions

Decision	What was chosen	Rationale
Session isolation via metadata, not separate tables	All sessions share one PGVector collection. Scope is enforced by a session_id field in cmetadata JSON.	Simpler operations: one collection to manage, one index. Trade-off: cross-session isolation is enforced by query filters. Defended by the post-retrieval session_id re-check in retrieve_v2.
BM25 in-memory with TTL cache	BM25Retriever is rebuilt from a full DB scan at most once per 600 seconds per session. Cache is an OrderedDict with LRU eviction at 20 sessions.	Avoids a separate search system (Elasticsearch, etc.). Acceptable because the corpus is expected to be small. Ingestion events flush the cache immediately; the TTL prevents constant rebuilds during query bursts.
session_id via AgentContext, not message context	AgentContext is a typed dataclass passed as context= to agent.invoke(). The retrieve_documents tool reads it via ToolRuntime[AgentContext].	Prevents the LLM from reading the session_id in its prompt and being tricked into routing to another session (prompt injection). The model only sees question text and tool results.
create_extension=False in PGVector	The pgvector extension is assumed to already exist in the database. PGVector will not attempt CREATE EXTENSION.	Creating pgvector requires superuser privileges. On managed Postgres services (Neon, Render), the extension is pre-installed. Setting this to True would cause startup failures on those services.
Cohere for reranking instead of a local cross-encoder	CohereRerank(model="rerank-v3.5") is used. The RERANK_MODEL config field ("cross-encoder/ms-marco-MiniLM-L-6-v2") is defined but not wired to any code path.	Inference: the config field may be a holdover from an earlier local-model plan or a placeholder. The current code imports CohereRerank only. This is a known inconsistency noted in Known Limitations.
ToolCallLimitMiddleware with exit_behavior="continue"	The agent may call retrieve_documents at most 3 times. On hitting the limit it continues to answer rather than erroring.	Prevents runaway tool-call loops without surfacing implementation errors to end users. The agent generates a best-effort answer from whatever it has retrieved.
Single-session app, no multi-user auth	No authentication layer. session_id is a client-generated UUID. Any client that knows a session_id can query or ingest into that session.	Auth is out of scope for a capstone portfolio project. Documented as a Known Limitation.
Chunk size 1500 / overlap 200	RecursiveCharacterTextSplitter with chunk_size=1500, chunk_overlap=200.	Sized to fit within the Voyage AI embedding window while preserving intra-paragraph

Decision	What was chosen	Rationale
		context. Larger than typical 512-token splits to reduce the chance that a single fact is split across two chunks.
Temp file pattern for PDF and DOCX	File bytes are written to a NamedTemporaryFile, the loader reads it, and the file is deleted in a finally block with a PermissionError catch.	PyPDFLoader and Docx2txtLoader require a file path, not in-memory bytes. The PermissionError catch handles Windows file-locking; on the Linux Render deployment this should never trigger.

5. Known Limitations

Area	Limitation	Impact
Authentication	No auth layer. <code>session_id</code> is a client-provided UUID with no verification. Any party knowing a <code>session_id</code> can read, ingest, or delete documents in that session.	High — not suitable for production multi-user deployment without auth.
BM25 scaling	<code>get_all_documents()</code> performs a full table scan on every BM25 rebuild. Fine for small corpora; expensive if document count grows into the thousands.	Medium — the code itself flags this in a comment.
Eval judge bias	RAGAS uses the same ChatGroq model as both the system under test and the judge. Self-evaluation is a known source of inflated scores.	Medium — scores are directional comparisons between v1 and v2, not absolute quality benchmarks.
Agentic eval gap	The v2+agent estimated scores are projections, not measurements. The eval harness calls <code>retrieval.retrieve()</code> and <code>generation.generate()</code> directly — it does not exercise the agent path (<code>run_agent()</code>).	Medium — a real eval requires adapting <code>run_eval.py</code> to call <code>run_agent()</code> .
.zip treated as Notion only	<code>LOADER_DISPATCH</code> maps .zip exclusively to <code>ingest_notion()</code> . A non-Notion zip will fail at <code>NotionDirectoryLoader</code> if it contains no Markdown files.	Low — acknowledged in a code comment. Only Notion exports should be uploaded as .zip.

6. Setup and Configuration

6.1 Prerequisites

- Python 3.13
- PostgreSQL with pgvector extension pre-installed (e.g. Neon, Supabase, or self-hosted)
- API keys: Groq, Voyage AI, Cohere

6.2 Environment Variables (.env in backend/)

Variable	Required	Notes
GROQ_API_KEY	Yes	Groq API key
DATABASE_URL	Yes	postgresql://user:pass@host/db — psycopg_connection_string() normalises to psycopg3 dialect automatically
COHERE_API_KEY	Yes	Cohere API key for rerank-v3.5
VOYAGE_API_KEY	Yes	Voyage AI API key for voyage-4-lite embeddings
ALLOWED_ORIGINS	No	JSON array or comma-separated list; defaults to ["http://localhost:3000"]
LLM_MODEL	No	Groq model string; defaults to openai/gpt-oss-120b
VOYAGE_MODEL	No	Defaults to voyage-4-lite
RETRIEVAL_STRATEGY	No	"v1" or "v2"; defaults to "v1". Set to "v2" to enable the full pipeline for all queries.
DEFAULT_K	No	Defaults to 7
INGESTION_BATCH_SIZE	No	Defaults to 100
BM25_WEIGHT	No	Defaults to 0.4
MULTI_HOP_MAX_SUBQUESTIONS	No	Defaults to 3

6.3 Dependencies

Install with `pip install -r backend/requirements.txt`.

Package (pinned version)	Purpose
fastapi==0.141.1	Web framework
uvicorn==0.53.0	ASGI server
python-multipart==0.0.32	Multipart form parsing for file uploads

Package (pinned version)	Purpose
pydantic-settings==2.15.0	.env file parsing
pytest==9.1.1	Test runner
langchain==1.4.2	Core LangChain abstractions
langchain-core==1.6.3	Document, BaseRetriever, messages
langchain-community==0.4.2	BM25Retriever, document loaders
langchain-text-splitters==1.1.2	RecursiveCharacterTextSplitter
langchain-groq==1.1.3	ChatGroq
langchain-classic==1.0.8	SelfQueryRetriever, EnsembleRetriever, LLMChainExtractor
langchain-cohere==0.6.0	CohereRerank
langchain-voyageai==0.4.1	VoyageAIEmbeddings
langchain-postgres==0.0.18	PGVector, PostgresChatMessageHistory
psycopg[binary]==3.3.6	psycopg3 — used by chat_history.py and the PGVector SQLAlchemy engine
psycopg2-binary==2.9.13	psycopg2 — for any LangChain internals that depend on it
rank_bm25==0.2.2	BM25Okapi algorithm underlying BM25Retriever
pypdf==6.19.0	PDF loading via PyPDFLoader
trafilatura==2.2.0	Web content extraction
fonttools==4.65.0	Dependency of pypdf for PDF font handling
docx2txt==0.9	DOCX text extraction via Docx2txtLoader

Eval harness (eval/requirements.txt) additionally requires ragas (unpinned).

6.4 Running Locally

- `uvicorn app.main:app --reload` — starts the FastAPI dev server on `http://127.0.0.1:8000`
- `http://127.0.0.1:8000/docs` — Swagger UI with all endpoints
- `pytest -v` — runs the test suite (mocks all external connections)
- `python eval/run_eval.py v1 --k 7` — runs the RAGAS eval for v1 at k=7; substitute v1/7 as needed

6.5 CI/CD Workflows

File	Trigger	Jobs
<code>.github/workflows/testing.yml</code>	Push to DEV; PR into DEV, v1, or v2	test: Python 3.13, pip install, pytest -v. Uses Preview environment secrets.
<code>.github/workflows/deploy.yml</code>	Push to main	test (Production secrets) → deploy: triggers Render deploy webhook if tests pass.

Both workflows use GitHub Actions. API secrets are stored in GitHub environment secrets. The test job injects dummy values — all external connections are mocked in the test suite, so no real API calls are made during CI.

Appendix A — Evaluation Dataset

The 15 questions used in the RAGAS evaluation harness, organised by source lecture. Each entry includes the expected source file and the reference answer as defined in eval/dataset.py.

Lecture 1 — Introduction to AI

Q1. What is the distinction between autonomous behavior and adaptive behavior in an AI agent?

Reference: Autonomous behavior means the system does not require constant instructions to operate, whereas adaptive behavior means the system can modify its behavior when the environment or problem space changes.

Expected source: CSE281_Lecture 1.pdf

Q2. What are the four levels of the DIKW Pyramid and how are they defined?

Reference: The four levels are: Data (unorganized, unprocessed raw facts), Information (aggregated data that provides context and answers questions), Knowledge (information combined with human experience and context), and Wisdom (the ability to make sound decisions using knowledge).

Expected source: CSE281_Lecture 1.pdf

Q3. What are the core differences between Supervised, Unsupervised, and Reinforcement Learning?

Reference: Supervised learning uses labeled training data with known outcomes; Unsupervised learning uncovers hidden patterns and structures in unlabeled data; Reinforcement learning trains agents through trial and error using rewards for desired actions and penalties for unfavorable ones.

Expected source: CSE281_Lecture 1.pdf

Lecture 2 — Search Algorithms

Q4. What are the five core components required to formally define a search problem?

Reference: A search problem is defined by an initial state, a set of available actions, a transition model $\text{RESULT}(s, a)$, a goal test, and a path cost function.

Expected source: CSE281_Lecture 2.pdf

Q5. What is the evaluation function of A* search, and under what condition is A* guaranteed to be optimal?

Reference: A* search evaluates nodes using $f(n) = g(n) + h(n)$, where $g(n)$ is the exact cost to reach node n and $h(n)$ is the estimated cost to the goal. It is optimal provided $h(n)$ is an admissible heuristic (it never overestimates the actual cost).

Expected source: CSE281_Lecture 2.pdf

Q6. When should Manhattan distance be used as a heuristic instead of Euclidean distance in grid-based pathfinding?

Reference: Manhattan distance should be used when movement is restricted to 4 directions (up, down, left, right), whereas Euclidean distance is appropriate when movement is allowed in any direction.

Expected source: CSE281_Lecture 2.pdf

Lecture 3 — Adversarial Search

Q7. What is the key structural difference between a simultaneous game and a sequential game?

Reference: In a simultaneous game, players make their moves at the same time without knowledge of the choices made by others, whereas in a sequential game, players take turns making moves and can observe previous actions.

Expected source: CSE281_Lecture 3.pdf

Q8. What defines a zero-sum game in game theory?

Reference: A zero-sum game is one where the total sum of payoffs across all players equals zero for every outcome, meaning any gain for one player results in an equal loss for another player.

Expected source: CSE281_Lecture 3.pdf

Q9. What general condition triggers pruning in the Alpha-Beta Pruning algorithm?

Reference: Pruning occurs whenever the alpha value at a node is greater than or equal to the beta value of an ancestor node ($\alpha \geq \beta$), meaning the current branch cannot influence the final outcome at the root.

Expected source: CSE281_Lecture 3.pdf

Lecture 4 — Knowledge Representation and Reasoning

Q10. How does declarative knowledge differ from procedural knowledge?

Reference: Declarative knowledge represents explicit static facts and information answering 'what' questions, whereas procedural knowledge represents step-by-step operational instructions and algorithms answering 'how' to perform tasks.

Expected source: CSE281_Lecture 4.pdf

Q11. How is knowledge structured inside a Frame representation?

Reference: A Frame encapsulates structured knowledge about stereotypical objects or situations using named attributes called 'slots' and corresponding values or constraints called 'filters'.

Expected source: CSE281_Lecture 4.pdf

Q12. What is the expressive advantage of First-Order Logic (FOL) over Propositional Logic?

Reference: Propositional logic treats basic facts as atomic propositions that are simply true or false, whereas First-Order Logic expresses complex relationships using objects, predicates, functions, and universal or existential quantifiers.

Expected source: CSE281_Lecture 4.pdf

Lecture 5 — Matrix Games and Logic Applications

Q13. What is a Saddle Point in a zero-sum payoff matrix game?

Reference: A Saddle Point is an entry in a payoff matrix that is simultaneously the minimum element in its row and the maximum element in its column, establishing the strict value of the game.

Expected source: CSE281_Lecture 5.pdf

Q14. What is the effect of removing a strictly dominated strategy on a matrix game?

Reference: Removing a strictly dominated strategy reduces matrix size without altering the overall value of the game or changing the set of optimal strategies.

Expected source: CSE281_Lecture 5.pdf

Q15. What steps should be followed to solve a 2x2 zero-sum matrix game?

Reference: First, test the matrix for a saddle point. If no saddle point exists, solve for the optimal mixed strategies by setting up an equation that equalizes expected payoffs across opponent moves.

Expected source: CSE281_Lecture 5.pdf

Appendix B — API Quick Reference

All endpoints are relative to the server root (e.g. `http://localhost:8000`). Trailing slashes are required on `/documents/` and `/query/`.

Document Endpoints

Method	Path	Auth	Request body / params	Response
GET	<code>/documents/</code>	None	Optional query param: <code>session_id</code>	<code>list[DocumentSummary]</code> — <code>source</code> , <code>source_type</code> , <code>session_id</code> , <code>chunk_count</code> per document
POST	<code>/documents/upload</code>	None	multipart/form-data: <code>file</code> (required), <code>session_id</code> (optional form field)	<code>IngestionResponse</code> — <code>source</code> , <code>source_type</code> , <code>chunk_count</code> , <code>sample_chunk</code> , <code>session_id</code>
POST	<code>/documents/web</code>	None	JSON: <code>{ url: string, session_id?: string }</code>	<code>IngestionResponse</code>
DELETE	<code>/documents/{source_name}</code>	None	Path param: <code>source_name</code> (URL-encoded; :path converter allows slashes). Optional query param: <code>session_id</code>	<code>{ detail: string, collection_id: string }</code> or HTTP 404

Query Endpoint

Method	Path	Auth	Request body	Response
POST	<code>/query/</code>	None	JSON: <code>{ question: string, version: "v1" "v2" (default "v1"), session_id?: string }</code>	<code>QueryResponse: { answer: string, sources: [{ source, source_type }], session_id: string }</code>

Health / Root

Method	Path	Response
GET	<code>/</code>	<code>{ message: "Welcome to the RAG Capstone API" }</code>
GET	<code>/health</code>	<code>{ status: "ok" }</code>

Error Codes

HTTP Code	When it occurs
200	Successful GET, POST, or DELETE
404	DELETE on a source_name with no matching chunks
415	Upload with an unsupported file extension (not .pdf, .docx, .zip)
422	File could not be parsed (empty PDF, non-Notion zip, trafilatura found no text), or request body validation failed
500	Embedding write to the vector store failed, or generation/retrieval threw an unhandled exception